

AI Express Hackathon — Technical Summary Sheet

Course Code: BCA 301-5

Group ID: 3

Members: Mayur V, K Prasanna, Shashwat Dwivedi

Selected Track: Track 3 — Legal Compliance Drone (Unit 4: First-Order Logic Agent)

GitHub Repository: <https://github.com/MORTAL8900912384/Legal-Compliance-Drone>

PEAS Framework Matrix

Performance Measure	Environment	Actuators	Sensors
Reach the delivery goal cell while never crossing FlyOver-denied airspace; minimise total moves (path cost) and number of replans.	8x8 discrete grid; partially observable (a zone's Restricted/Permit status is confirmed only via an FOL query at its boundary); static rule set, deterministic, single-agent.	One-cell move (up/down/left/right); hold/pause at a zone boundary while a query resolves.	Position sensor (current cell); zone-boundary detector (triggers an FOL query); knowledge-base reader (Restricted / Permit facts).

Core Algorithmic Formulation

State space: $S = (\text{cell}, \text{blocked_zone_set})$ where cell is the drone's grid position and blocked_zone_set is the set of zones so far denied by the FOL engine. **Initial state:** (Start cell, $\emptyset$). **Goal test:** cell == Goal cell. **Path cost:** uniform, 1 per grid move (4-directional). **Path planning:** Breadth-First Search (optimal under uniform cost), re-invoked from the drone's current cell whenever a zone query is denied (dynamic replanning), with the denied zone's cells added to blocked_zone_set as obstacles.

FOL rules (Horn-clause form, executed by a generic unification-based engine supporting both forward and backward chaining with negation-as-failure):

R0 Drone(d) $\wedge$ Zone(x) $\wedge$ $\neg$ Permit(d,x) $\Rightarrow$ NoPermit(d,x)

R1 Restricted(x) $\wedge$ NoPermit(d,x) $\Rightarrow$ $\neg$ FlyOver(d,x) (the rule given in the brief)

R2 Drone(d) $\wedge$ Zone(x) $\wedge$ $\neg$ Restricted(x) $\Rightarrow$ FlyOver(d,x)

R3 Restricted(x) $\wedge$ Permit(d,x) $\Rightarrow$ FlyOver(d,x)

Backward chaining (SLD resolution) answers each live boundary query FlyOver(Drone1, Zone) goal-directed, with a full proof trace.

Forward chaining runs once pre-flight to derive the complete permission table over all known zones.

Complexity Analysis

Component	Theoretical	Observed (this run)
FOL backward-chain query	$O(b^d)$, b=rules/facts matching a goal, d=max rule-chain depth (d=2 here) — effectively $O(1)$ for this finite, acyclic, function-free rule set	2 queries, each resolved in <5 proof steps, <1 ms
FOL forward chaining	$O(\text{rules} \cdot \text{facts} ^k)$, k=max body literals (k=3) per fixpoint pass, bounded by finite derivable fact set F	3 new facts derived in 1 pass over 4 rules
BFS path (re)plan	$O(V+E)$ per plan on an RxC grid = $O(R.C)$; total = $O(\text{replans} \cdot R.C)$	1 initial plan + 1 replan, 89 total nodes expanded on an 8x8 grid (V=64)
End-to-end mission	$O(R.C)$ dominates (grid search over shallow, constant-depth logic queries)	13 moves (path cost), ~13-20s wall-clock depending on animation pacing

Re-run `python main.py` and read the Mission Metrics panel to reproduce/verify the observed figures above.